Video Transcripts

Module 22: Processing and Streaming Big Data

Video 1: Introduction (04:10)

So, this week we're going to be looking at really big data. We've been looking at files that are a few 1,000 lines long, and we've been reading in CSV and JSON formats. This week we're going to read a file that's over 100 million lines long and we're going to see that pandas takes quite a time to read a file that large. And so, we're going to look at a couple of other formats, Parquet, and Feather, and HDF5 are other formats that are actually faster. And we're also going to look at our machines in some detail because our machines these days have multiple cores and we can run lots of processes and threads on those cores. If you look at your laptop and see how many threads it's running; I looked at mine, it was running around 3,000 threads.

So, we need to understand the difference between cores, threads, and processes, and we need to understand how to exploit those that we can run our programs in parallel. So, we're going to use DASK, which is a Python library which allows us to do things in parallel. We're going to look at, first, writing 20 files at a time. And, then we're going to look at how can we read back those files in parallel. We're going to explore DASK, D-A-S-K and we're also going to look at a library called Graphviz, which shows us our execution graph, how our functions are going to be called and executed. Today, most computation is done in parallel. We're taking advantage of machines with 64 cores, 128 cores in some cases.

So, we can do a lot in parallel, and we probably need to do that. So, after we've looked at big files, we'll be looking at really big files, files that are too large for us to actually read into memory. This comes under the heading of streaming data. Often if we're, for example, monitoring mouse clicks on our website, we might be getting a billion events a day being streamed at us and we need to develop a strategy for handling streaming data. We'll start by looking at web sockets. So far, we've been using HTTP and that has a request response built into it, that it's not as if either party can send messages to the other. One party makes a request to the web server and the web server responds. Now, in the case of web sockets, either party can communicate and they can

© MIT xPRO® 2024. All rights reserved.

communicate in parallel.

So, while you're sending me data, I could be sending you data. So, web sockets are extremely efficient when it comes to handling streaming data. And we're going to do an exercise where you're going to look at how web sockets handle chunks of data and how we implement on top of those TCP packets, the idea of a message. So, we're going to handcraft our own messages using web sockets. And, this will set us up then for next week, we're going to look at streaming data, MQTT, and Kafka. But for now, let's take a look at really large files and setting up web sockets.

Video 2: Using Pandas to Read Big Data (02:50)

Now, we know from manipulating files, it's really nice if we can read into a Pandas data frame. So, let's take a look. We know we can read CSV files in Panda, so let's load up. We'll import Pandas as we normally do. We'll put our timing in here. Now, I happen to know the data types that the columns are using in the file, and I'm putting those in so it'll actually read it faster. So, we're going to read the file in as a CSV file, but there's perhaps a better way of handling these files.

And, Parquet is one. So, once we've got it read in, I'm going to write it out to Parquet. Maybe I'll call it my million lines. Let's write it out. So, example02, and we'll see how long this takes. I'm going to stop the video just to show you that we're using 3.38 gigabytes of memory at the moment here in our Python code, so it's quite voluminous. This is a four gigabyte, it went up to five quite suddenly, and we finished, excellent. So, we read in 101 million lines, and we did that in 187 seconds.

So, Pandas is not fast but it can read 100 million lines. That'll give you some idea of the size file. And now we wrote out our file, and this case we chose Parquet format and that's a better format for reading files in. Okay so, I wrote the file out for you in Parquet format, it's called riid_train.parquet. Why don't you read that back in? Tell me how long it takes to read then, write it out in a format called Feather. So, instead of Parquet here so, just replace it with Feather format and let me know how long that takes to read back in. Okay so, that's your exercise.

Video 3: Exploring Threads (06:23)

We've seen that DASK allows us to run tasks in parallel. So, we looked at two functions: increment and add. And we saw that increment could be executed in parallel while add had to be

© MIT xPRO® 2024. All rights reserved.

executed sequentially. So, DASK comes up with a task graph, and then, that's passed to a scheduler. But, how is this executed? Here, it says that it scales up and it can run on thousands of cores. Let's take a look at a modern machine. So, here's the chipset, and here's an example of a four-core machine. So, this has four CPUs, if you like, central processing units, each of these can execute instructions. Each core can be passed separate instructions. Each core has access to its own cache, L1, which is very fast. Just a few clock ticks, three or four clock ticks, to access L1 cache. L2 cache is typically 14 or so clock ticks, and then L3 cache can be 60 or 70 clock ticks, and RAM can be hundreds or thousands of clock ticks.

So, if we want things to run very quickly, we need to think about how they're executing, how much cache they're using, and how many cores we have on the machine that can execute. Now, we talk about threads, and one way to think about it is that a core operates on a set of instructions that are passed to it on a thread. Now, often these instructions might be to go out and read a file, and that may take millions of clock ticks just to go out onto the web and retrieve a file. So, during that time the core can execute another thread. So, typically processes have multiple threads. So, here I'm showing process one has three threads. Now, threads can also have priority. If, for example, they belong to the operating system, they're going to have high priority, whereas, some of our tasks have low priority. So, threads can execute alongside each other, but they may have different priorities. And the time slicing that each thread gets will depend on its priority.

So, here I'm showing different threads being passed into. Here, we've got the four cores. Let's go and take a look at some code. So, here I've got a program, it's called threads.py. We've got a function that's being called and that function sleeps. I'm going to have it sleep for three seconds, and let's spin up five threads. So here, this loop creates a thread, appends it to our list of threads, starts that thread and then when it returns, it actually joins it back into the main thread. So, let's see this running. And if we look here on the right, we'll see these threads, see them running, then they're gone. Let's have them sleep a little longer. Let's have them sleep for 10, so we can see them.

So, we're executing. And now, the threads don't start with Thread-1, but we've got five of them, and then we'll see them finishing. And you'll see over here, we had zero done, then four was done, then one, then two, then three. So, although we launched them in an order, they don't necessarily return in that order. They get arbitrary amounts of time slices that the operating

© MIT xPRO® 2024. All rights reserved.

system decides on, and they may finish in a different order. So, you've got to be careful when you're launching threads; don't expect them to come back with their results in the same order that you launch them. We typically talk about a scatter-gather strategy. You scatter your computations out to these threads and then you gather them together to synchronize them again.

Let's take a quick look at, on my computer here, my activity app. Here, we can see this is CPU time. Here, you can see the number of threads. So, the operating system, my kernel, has 231 threads; MySQL, 38 threads. The code we just ran, 30 threads. So typically, we literally have thousands of threads running on our machine. So, why don't you take this code and why don't you spin up more threads by changing this number? And perhaps you want to have them sleep for different amounts of time. So, there's a parameter x is passed into test, and this is i being passed in. So, depending on which thread it is, we can have it sleep for a different amount of time. Okay, bye for now.

Video 4: Introduction to DASK and Running Parallel Operations (09:28)

We're going to start using DASK for parallel computing. Let's take a look first at some simple tasks and timing them. So, for example, let's suppose we want to time the execution in this cell. We can put in time, and let's do from time. Let's import sleep, and let's have a couple of functions. So, let's define increment, and we'll pass in an argument. And let's just sleep for one second, and return the result, x + 1. Okay, and let's do add, and add will add x and y. And again, we'll put in sleep for one second just to exaggerate the timing that each of these takes to execute. And, we'll return x + y.

Let's say x equals increment and we'll pass in one, and y equals increment, pass in two, and z equals add x and y, okay. And, we'll print z out. Okay, let's execute this, and we'll see that this takes three seconds. And it takes three seconds because to calculate the first x here, this increment call will sleep for a second, this one will sleep for a second, and this one again will sleep for one second. But, we could do these x calculation and y calculation in parallel, they're independent. So, now let's import from DASK import delayed.

And what we're going to do is we're going to put wrappers around these. So, let me copy those. We're going to put the decorator around this delayed, and we'll see exactly how it works, but it's going to automatically parallelize for us this calculation, delayed.

© MIT xPRO® 2024. All rights reserved.

Okay so, let's calculate x equals delayed, and we're going to pass in increment and pass into it a value. And y equals delayed, pass in the function and the value two, say. And now z equal, let's call add this time, and here we'll pass in x and y. And now, we need to call compute because this actually won't do anything. Let me execute it and show you, let's put a timer in here. So, let's time this. So, it took one millisecond, and that's because this hasn't actually done the calculation yet. It's just setting it up.

And what do I mean by setting it up? Well, if we put in visualize, we'll see that. So, let me take out time here and run it with visualize. And we see that it shows us the graph of how it's going to do this calculation. So, it does the increments in parallel, and then, we'll go to the add. And, let's take a look at the time for it because what do you guess the time should be if it's doing the increments in parallel? But, the add. Now, we have to be careful because it hasn't actually done the calculation. To do the calculation, we need to tell it to compute. Let's run it, and you see that it takes two seconds.

So, before it took three seconds; now it's doing things in parallel. So, let's take a look at another operation that takes time. So, let's suppose we've got an array, and we've got a for loop that we're going to operate on the array. We'll still have the increment, so let's put that back. And now, let's have a for loop, for i in data. So, we're looping over that. Let's get ourselves a result array. There it is, and now, we're going to append to that. We'll increment i, so we've got that now, result, and let's get the total as the sum of the result. Okay, so let's see how long this takes. So, it took six seconds. So, each call in the for loop to increment took one second. Now, we can do better than that because we can do things in parallel. So, let's go back and we'll add our decorator. And now, here in result, we'll add that. And in this, let's delay this as well. And passing result, and let's print out the total. Okay so, this should now be all in parallel. Let's see what our timing is now. Remember, we didn't execute anything yet. We've got to do compute. So, three seconds.

So, this iteration, the whole iteration only took one second. But this delayed in the result, probably took a second as well. Now, let's take a look what was actually going on. Let's visualize it and see what it did. And we need to take this out. So, this is what actually happened in the computation. All of these were done in parallel, went up to the sum, and then that was the total. So, DASK does an amazing job of parallelizing operations, and we're going to see that later when we're operating on big data in lots of files. But that's an introduction to DASK and actually to the

© MIT xPRO® 2024. All rights reserved.

visualization. It's using graphviz to do this visualization. It's very neat indeed. Okay, bye for now.

Video 5: Using DASK to Create Multiple Files in Parallel (04:16)

In this we're going to take advantage of DASK's multi-processing ability. We're going to be able to create multiple files in parallel. Let's take a look at what we've got. Here, I've got a Python file, I'm importing DASK, os, and datetime. We're going to create a directory data. So, I'm sitting here under a directory called DASK and I have just the one file dask03.py. We're going to take advantage of DASK's datasets time series to create basically 30 different, a data frame that has 30 components in it. And these are going to be the files. We've got a small function here that's going to increment and create a datetime.

And the structure of these data files is going to be a time series. So, we're going to increment by one, the days, and we're going to go in increments delta time of one. So, we're going to get up to 30 days in 2000, 1, 1, 2000, January of 2000. Okay, so let's run this. So, I'm going to go into run and debug. We'll choose this file. We've only got one. And now we'll see here we have multiple threads. So, let's move this up a little bit and we can see main thread and then 1, 2, 3, 4, 5, 6, 7, 8.

I've got 8 cores on this machine and they're all running. And if we take a look, now we've got a directory, data. Let's see what we have in data. We'll see, we have 30 files in data. So, we've created those files, each file. So, we'll see each file here. This is the file size, so each file is around 6.1 megabytes. So, there's quite a lot of data in each file, and we've got 30 of them. And we can take a look at the content of those files. So, here we've got our data directory. So, here's our files.

So, we see that we go up to, from 1 to 30, we have 30 files. Let's take a look in one of the files. So, here we've got a timestamp, an ID, a name, and an XY coordinate and it automatically gives different names and different IDs, and the coordinates are given too. So, you can see this is quite a long file that it has 85, 86,000 entries. Let's have a look at the bottom of the file, 86,401. So, that's the data we've created and we've got 30 files and let's see if we can go and read it. Okay, why don't you create your data files and follow along? Okay, bye for now.

Video 6: Using DASK to Read and Analyze Multiple Files in Parallel (05:41)

So, we saw how to use DASK to create 30 files in parallel. Each of these files was 86,401 lines long. And if we look at the columns, we have timestamp, id, name, X, and Y. So, now let's see

© MIT xPRO® 2024. All rights reserved.

how we can read these. So, here was our code for writing them and we wrote them in parallel. And so remember they all start with 2000. So, let's take a look at reading them. So, here we're going to import DASK dataframe as DDF. Import DASK delayed, we don't need to import that.

And so, here we're going to read in parallel. So, we do ddf.read_CSV and we give a wild card; the star. So, any of the files starting 2000*. So, there should be 30 of those, and then we can print the head.

So, let's take a look at running that first. So, here, let me clear this and we'll run Python desk 04. So, you'll see here that it's printed out the head. Now, the head of what? What is it read in? Let's take a quick look. So, what it's read in, is a DASK dataframe. And a dataframe consists of multiple Pandas dataframes.

So, each of our dated files is going to be a Pandas dataframe. Now this whole area, you'll need to dig in a lot deeper, but the DASK reads and writes are very similar to the Pandas reads and writes. So, let's go back to our code. Let's now do some calculations on what we've read in. So, let's take a look now at printing out, for example, let's get the mean of the X column. So, first we need to do a df.compute to make sure that we've actually got a dataframe. Remember, this is a DASK one.

Now we can do, for example, we know there's an X column, so that'll give us the X column and we could calculate the mean and we need to do compute. Now, let's get the result mean and print it out. Okay, that looks good. Let's save it and let's run it. So, notice now it's taking a little while, but it's calculated the mean. And we can similarly calculate the mean of the Y column. Let's take a look how, just to make sure, what our data structure looks like now. Let's take a look at the number of columns.

So, we can do df.columns and print that out. And let's do the rows as well. So, let me, I'm just going to copy and paste this and instead of columns, we'll have rows and here instead of that we need the index because that'll give us the rows. Okay, let's take a look and run it. So, we've got five columns and we've got 2.59 million rows. So, you remember that we had around 86,000 rows in each of those 30. If you do the multiplication, you'll come up with 2.5 million. So, we're handling fairly large data files here, certainly in the millions. So, that's reading using DASK and DASK's parallel computing powers. So, one, you repeat what I've done here so that you understand just how this works. OK, bye for now.

© MIT xPRO® 2024. All rights reserved.

Video 7: Using DASK for Parallel Processing Across Distributed Machines (04:07)

So, let's take a look at DASK running on different machines. I'm going to simulate different machines on my single machine here, but let's first, we're going to launch the dask-scheduler. So, this tells us which port it's running on. So, the scheduler is running on this port. So, what we'll do now is set up two workers. So, it's 8786. So, now we've got a dask-worker connected to the scheduler.

Let's run another dask-worker and connect to the same scheduler. So, here we are now. So, we've got two of them. Now, let's get a client here. So, we need to run Python. So, here we need to say 'from dask.distributed import Client.' And now we say 'client' equals 'Client' and we need to paste in address. And now, we've got our client connected. You see here, we've got our client talking to the scheduler. Now, we can put our program into here.

So, let's type our program. So, we need to import dask.array as da. Now, we'll set up our variables so, da.ones and we'll set up 100,000 by 100,000. Okay, and I need an extra bracket in here.

So, we've got X set up. Now, we can run our calculation. Y equals da.exp(x).sum. Now, dot compute will spread the computation now across these machines, these workers. So, this normally takes about 25 seconds, and we've got the result. I can check that, but that's correct. So, this is a very nice distributed computing environment. DASK allows us to communicate across machines, basically distribute our computation in a parallel fashion. So, let's just take down the scheduler and we'll see that the other machines. So, let's just control C and notice the other machines, notice the connection is broken and they'll eventually log out. There we go. They're all logged out. Okay, so that's an exciting environment to program in. Bye for now.

Video 8: Introduction to Web Sockets (04:29)

Let's talk about web sockets. We've been using HTTP which runs over TCP networks, but web sockets also run over TCP networks. Now, web sockets provide a persistent connection, HTTP doesn't. That HTTP is a request response protocol. TCP sets up a persistent two-way connection, and either or both parties can then send messages. Now, it requires both parties to set up a TCP connection, but either of them can then take it down. It's heavily used where we have streaming data, for example, in IoT where we have sensors. Here, we're showing a number of sensors. For

© MIT xPRO® 2024. All rights reserved.

example, perhaps a drone is sending video feed, and that a number of clients may be interested in receiving data from those feeds.

So, web sockets is often used in chat messaging apps. It's a lighter protocol than HTTP, so it doesn't have so much in the headers. And it's preferred for, say, IoT networks where we're sending a lot of streaming messages. And we've seen how to handle large files of over a hundred million lines. But, how about streams of data that continue for weeks, months, or years? How do we handle those streams of data? Well, web sockets can help us. It's not the only one. We're going to look at another one called MQTT, but web sockets is widely used.

So, just to remind you about TCP networks. So, TCP packets have the address that they're going to be sent to, and it's an IP address plus a port number. And it has an IP address of where it's being sent from and also the port number there. Now, a machine has many processes listening on ports, and some of those ports may be listening for TCP messages, but there are also other message types, UDP, FTP, DNS, etc. So, we have these apps continuously listening on these ports, and here's some of the other protocols that you might see, tcp, tcp6, udp.

If you do netstat in a terminal window, you'll see who your machine is talking to, and you'll be surprised the number of communications it's got set up. Now, the TCP messages are coming in a stream, and they're directed to certain ports. Now, if it's a TCP port, then it will have a buffer. And the packets are not in order as they arrive, but the facilities on your machine will order those TCP packets in the TCP buffers. Now this is going to cause us some issues because the web socket protocol doesn't have the concept of a message.

So, the stream of packets are going to be coming in, they'll fill up the TCP buffer and when we make a web socket request, a read for example, it'll read the whole buffer. So, we're going to have to handcraft into the web services protocol, the concept of a message, so that we know where messages begin and where they end. We need to know when are we done with this message, and when do we need to wait, and then read another message. So, let's take a look at setting up a web socket messaging system where we have persistent connections, and we have streams of data that may be running for days, or weeks, or months. Okay, let's go and code messages into web sockets. Bye for now.

Video 9: Video 9: Web Sockets Streaming Exercise (15:52)

© MIT xPRO® 2024. All rights reserved.

So, we've been looking at web sockets and at how we can stream data from IoT and other devices. And the problem that we came up against was that the web sockets has no concept of a message. So, here we've got a server that is sending messages. Let's assume for now that the connection, the web socket connection has been set up so the server and the client are both willing to receive and send messages. The server is sending messages, and suppose it sends two 10 byte messages, so there are two sends. Now what actually goes out over the network is 20 bytes and that's going to be buffered and the client will make a read, a receive, and will get 20 bytes.

So, there is no break in the TCP packets that tells us that we're at the end of a message. We're going to have to put that in and we've got a couple of choices. One way is to have fixed length messages that we say every message is going to be 50 bytes, but that really won't work too well. So, one of the best solutions is that we're going to have a fixed length header at the beginning of a message. So, what we mean by that is we're going to choose a certain number of bytes and we can make it as long as we like, but that will be the number of characters in the message. So, we could set this to a billion and we could have a billion characters in our message.

Now we're not going to go that big, but that's the idea. So, we're going to send the message and we'll start reading through that message. And once we've read those 11 characters, we'll know we're at the end of the message. So, this is our strategy, fixed header size, and then the remainder of that will be 11 characters. So, we know we've got a certain length here, we're going to choose 11 for this length, and then the rest will be the 11 characters. Now when we get the message, it may not come to us as a single chunk, if you like.

We can decide how many bytes we want to read in our chunks. And so, we may end up with something like this that we have 31, then the Hello World, It's a good day. But then we get the beginning of the next message in that chunk. And so this is going to be an issue for us. Let's go and take a look at some code and then come back in a little more detail to this. So, here we are, and we've got some clients and we're going to set up first, the server that's going to send these messages.

So, we're going to choose the header for how long a message is. This is setting up the web socket, so we need to import socket. I'm importing time as well because we may want to put some sleep spaces, if you like, between these messages. And then, we're going to keep this web socket alive.

© MIT xPRO® 2024. All rights reserved.

So, we'll have while True and that's going to stay active for potentially forever. And we're saying we can accept connections. So, we're the server, we're going to sit there and if somebody contacts us, we'll accept that connection. We're going to send them the first message is going to be, "Hello from the Server- You are Connected".

And then we're going to have a loop where we're going to send them the time is and we're going to push the time into that message. So, what we're doing here is we need to calculate the length of the message. Now that length could be a lot of digits and we've got to make sure it's less than 10 digits because we can only go up to 10 because we chose header to be 10. So, what this is doing is putting that into a string and then padding it out to the length of the header with nothings.

So, this is going to be for example, this length of the message might be 31, so that's three and a one. So, now we've got another eight blank characters up until the end of the header, which is 10 long, and then we add the message on as a string as well. Now, we send that in bytes and using 'utf- 8' format to the client, and here we're just printing out the message so we can see it. So, here on this side, we're just zipping out a stream of messages. So, we might want to put asleep between these to slow them down. But essentially, that's what's happening. Now, let's take a look at the client side.

We need to deal with messages coming in, and we're going to decide to receive them in chunks, so we can specify how long this chunk is, how many bytes. Now the problem is that these chunks will not be related to the message lengths. So, we'll have part of a message here in the first chunk. Then we may have another part and another part, and eventually, we'll have the end of the message. So, this vertical line here, but we might have the beginning of the next message. So, here I'm showing 29, and maybe, there's a blank here because we know the header is 10 characters long.

So, this is what we've got to deal with. We're reading chunks, but we need to understand when the message is complete. And we need to deal with this end part and put it into the beginning of the next message. Let's take a look at our code and we start importing socket and set the header. Now we'll connect to the port on our local machine and it's 9092 on mine. You can choose, as long as you're the same on the server and the client, you'll be fine. Now, here we're reading a chunk, so we're setting up full message as blank at the moment. We're going to fill that up until

© MIT xPRO® 2024. All rights reserved.

we've got at least the full first message and maybe more of the next message. Let's suppose we're dealing with the first message.

So, here we read the first chunk. Now we've got to get the length from the header, the length of the message. So, I might have something already in first message, but if this is the very first message, this is going to be a blank, and we'll just have what we've read in here, that chunk. And so now, we can get the length of the message by reading from zero to the header and converting that to an integer and that'll give us the message length. So, if it's a new message, then we're going to re-get the message length.

Now, we're going to add the chunk to the full message. We're going to check if we've got the whole of that message. So, if the length of full message is greater than the message length plus header. because the header isn't in this message length, we know it's 10. But if the full message is less than that, we're going to read another chunk. But once it's greater than that so now, we're going to print out from zero up to the message length plus header, up to there. So, if the message length is 29, we're going to read up from zero to 29 plus 10, 39. Now, the start of the next message may be in that last chunk, and I'm not dealing with it here.

So, that's your challenge. You've got to deal with that next bit. So, let me show you where you we are that we are at this stage where we've got the complete message, but we may have this part. So, what we need to do is to run along in the full message up to this point and then save this. Now what do we do with that? Well, what I suggest is that we now print out this full message. So, from here to here, and now we save this next bit. Let's call it next. Whatever's left here in this chunk, we put that into the start of the next message, and we may as well use full message because we've printed out the last full message. Now,

I'm starting the next full message, so I'll put that into full message and then we'll continue reading chunks. We'll go back to the beginning again. Let me show you how the correct version runs. So, here I've got two windows. My code is in here, so I'm going to set up the server. So, we run stream server and that's running. Now we go and we run. You need to run your solution. I'm going to run my solution. So, stream client one solution. Let's see what happens. So, the server served up all the messages and here we are, we've caught them all. You can see that all the messages have been received by the client right up to the last digit and printed out beautifully.

So, that's what I want you to be able to do. Don't worry about these Ms. or Cs, that's me with

© MIT xPRO® 2024. All rights reserved.

some little bit of debugging. But what you'll need to do is to debug this code. So, this is going to throw an error. So, let me break out of this. You may have some problems with finding that when you restart that it tells you that the port is being used. But I found if you break in the client first and then in the server, do control C in the client first and then in the server, you won't have that problem.

So, I've started the server up again. Now, I'm going to run this time in debug mode here. So, I'm in your starter code and let's put a, let's put a break point in, for example, when we're we've got one there. Good. So, here we go. So, we've read in a chunk, you see it's got the 29 and it's got the rest of the header that's blank and then it's got the 'ti'. So, we've got that. Now, we need to get the length of the message. Yes, we've got 29 and that's what we've, that's fine. Now, we're going to add the incoming chunk to full message and full message is blank at the moment. But when we've added that chunk to it, now full message will be 29, the 'ti'.

Now, we're checking if the full message is longer than the message length, that's 29 plus header, that's 10, 39. Is the full message longer than that? No, it's not. So, we go back and we read another chunk. So, this is how you need to go about debugging is looking at where your code is, what it's doing, reading another chunk now. So, I've read three chunks. It's not a new message. So now, we're going to have that, this full message length. You can see it here and it looks like it might have a 30 that belongs to the next and some spaces as well. Let's take a look.

So, it certainly has come into here. We've got at least the first message printing it out there it is. Now the problem is I didn't deal with this next bit that's actually on the end of this full message. If we look at full message at the moment, notice it's got actually here that 30 and the spaces belong to the next message. I happen to know that. So now, we're going to come around, read another chunk, but so that's got the time is, but notice it doesn't have a header of any kind, it doesn't tell us the length. And that's because there's an error with my code. So, I want you to correct this. I'm giving you some hints down here about what you need to do. You don't need to change anything above this, but you can if you want. But make sure that you can write out the messages. But if you're in debugger, you can control the reading of the messages and step through and deal with this problem that we have the start of the next message embedded in the chunk somewhere. Okay, so good luck with this exercise. Bye for now.

Video 10: Web Sockets Streaming Exercise Solution (04:19)

© MIT xPRO® 2024. All rights reserved.

So, if you remember, we needed to solve the problem of the web streaming exercise where we had a web socket server that was streaming data to the client and we needed to deal with reading chunks on the client so we could specify how big a chunk was. And the problem is that we decided to put a header in where we'd put the length of the message so that the end of the message wouldn't be guaranteed to be at the end of a chunk. So, we'd need to make sure that we're monitoring, what we might call, the full message that we've got so far, and we need to check its length.

So, for example, if this was 31 long, we know the header is 10 long, so the full message length would be 41. And so, this would be the 41 break. And we'd need to deal with this start of the next message, which might be something like 29 and maybe some blanks because that needs to be tagged on and make sure that it's the beginning of the next full message. So, I gave you some clues as to how to do that. Let's look at the code. So, this is the code I gave you, and the last thing we're doing is we're printing out the full message from zero to the message length plus the header.

Now, the full message contains maybe more than that, and that piece we need to put into the next message. So, let's call the next message and it's going to be full message, and we're going to go from the message length plus header. So, we're going to go from the end point there to the very end of that full message. Now, suppose this is actually the end of the message. Suppose this is the end of the message here. Will this cause a problem? No, no. So, what I'm saying is, even if 'next' contains nothing, that's not a problem.

Now, we want to put that as the start of the next message. So, now we just replace the code here. This, actually, let's put that with, that's 'next'. So, let me get rid, now, of all these comments so we can see the code. So, now this should be our solution. So, we're reinitializing full message. And in fact, we don't need 'next', that we could eliminate it, but I'm keeping it there just for clarity that this is the start of the next message. Now, we reinitialize full message and go back now to reading chunks.

So, let's save that and let's go to our Windows. Let's break into here and break into here. It's clear. Okay, and let's run, now, our server and let's run our solution. So, I changed your client, so we should be fine. That runs beautifully. So, this is our client and it mimics this beautifully and we get the end of the last message. Okay, so I hope you enjoyed that exercise.

© MIT xPRO® 2024. All rights reserved.